COURSE SUBMISSION • DUE AUGUST 17, 2026

CareNavigator PH

API Integration & HTTP Requests (Week 5)

CRUD Functionality (Week 4)

Document order: Week 5 is presented first. Week 4 begins after a dedicated divider page so the two submission sets remain clearly separated.

Project type: Flutter web/mobile healthcare navigation application

Backend: Supabase Data API, PostgreSQL, Auth, Realtime, Storage, and Edge Functions

Prepared: August 15, 2026

Student: _____

Course / Section: _____

1. Live GET evidence from the configured Supabase project
2. HTTP POST implementation with validation and JSON processing
3. Database-backed Create, Read, Update, and Delete for doctor schedule slots
4. Reproducible UI screenshots and passing verification results

PART I • WEEK 5

API Integration & HTTP Requests

External/custom backend API integration for dynamic care-directory data and consultation workflows.

API name: Supabase Data API (PostgREST) with the CareNavigator Edge Function API. The integration retrieves verified hospital data, creates consultation requests, and supplies the care assistant with structured JSON.

API endpoints and methods

Method	Endpoint	Purpose
GET	`/rest/v1/hospitals`	Retrieves verified, open or limited hospitals for the public directory.
POST	`/rest/v1/guest_consultation_requests`	Creates a validated consultation request and returns the new record ID.
POST	`/functions/v1/care-navigator-chat`	Sends message/facility JSON to the care-assistant Edge Function.

Features added through the integration

5. A live public directory of verified Philippine hospitals, including operating state and availability information.
6. Dynamic display of API data in Flutter rather than hard-coded records.
7. Validated consultation-request creation through a POST request.
8. Care-assistant JSON exchange through a protected Supabase Edge Function.
9. Explicit loading, empty, unavailable, retry, and malformed-response handling.

WEEK 5 EVIDENCE 1

HTTP GET request implementation

Figure 1. Supabase query used to retrieve verified, operating hospitals.

HTTP GET request - Supabase hospital directory

lib/src/repositories/hospital_repository.dart

```dart
final class SupabaseHospitalRepository implements HospitalRepository {
  SupabaseHospitalRepository(this._client);

  final SupabaseClient _client;

  @override
  Future<List<HospitalDirectoryEntry>> loadPublicDirectory() async {
    try {
      final hospitalRows = await _client
          .from('hospitals')
          .select(
            'id,hospital_name,address,city,province,latitude,longitude,contact_number,
                emergency_contact_number,email,description,image_url,operating_hours,operating_status,
                updated_at,verification_status,hospital_classifications(classification_name)',
          )
          .eq('verification_status', 'verified')
          .inFilter('operating_status', const ['open', 'limited'])
          .order('hospital_name');
      if (hospitalRows.isEmpty) return const [];

      final hospitalIds = hospitalRows
```

The Supabase Dart client composes an HTTP GET request to the hospitals REST endpoint. The query selects required JSON fields, filters verification_status to verified, restricts operating_status to open or limited, and orders results by hospital_name.

Meaningful processing: The repository immediately returns an empty list for an empty response, then joins related departments, services, emergency-room records, bed capacity, doctors, and schedules before creating typed UI models.

WEEK 5 EVIDENCE 2

HTTP POST request implementation

Figure 2. POST-style insert for a validated guest consultation request.

HTTP POST request - guest consultation creation

lib/src/repositories/consultation_repository.dart

```dart
    try {
      final data = await _client
          .from('guest_consultation_requests')
          .insert({
            'submitted_by': authUser.id,
            'first_name': draft.firstName.trim(),
            'last_name': draft.lastName.trim(),
            'full_name': draft.fullName.trim(),
            'birth_date': _date(draft.birthDate),
            'sex': draft.sex,
            'mobile_number': draft.mobileNumber.trim(),
            'email': normalizedEmail,
            'address': draft.address.trim(),
            'symptoms': draft.concern.trim(),
            'symptom_duration': draft.symptomDuration.trim(),
            'consultation_reason': draft.concern.trim(),
            'preferred_hospital_id': draft.hospitalId,
            'preferred_department_id': draft.departmentId,
            'preferred_consultation_type': draft.consultationType,
            'preferred_schedule': draft.preferredStart
                ?.toUtc()
                .toIso8601String(),
            'otp_verified_at': DateTime.now().toUtc().toIso8601String(),
            'consent_at': DateTime.now().toUtc().toIso8601String(),
          })
          .select('id')
          .single();
      return data['id'].toString();
    } on PostgrestException catch (error) {
```

The repository sends a JSON request body to guest_consultation_requests, requests the inserted id, parses the returned object, and exposes the ID to the calling UI. Authentication and field validation run before the request.

Validation: Required fields, email format, concern length, hospital/department selection, supported consultation type, future schedule time, and verified-email ownership are checked before data is sent.

WEEK 5 EVIDENCE 3

JSON parsing and processing

Figure 3. JSON rows converted into typed HospitalDirectoryEntry models.

JSON parsing and application model mapping
lib/src/repositories/hospital_repository.dart

```dart
            final emergency = emergencyByHospital[hospitalId];
            final operatingStatus = row['operating_status']?.toString();
            final erStatus = emergency?['status']?.toString();
            final latitude = _doubleValue(row['latitude']);
            final longitude = _doubleValue(row['longitude']);
            return HospitalDirectoryEntry(
              id: hospitalId,
              name: row['hospital_name']?.toString() ?? 'Hospital',
              city: row['city']?.toString() ?? '',
              province: row['province']?.toString() ?? '',
              careLevel:
                  classification?['classification_name']?.toString() ??
                  'Hospital',
              services: services[hospitalId] ?? const [],
              departments: departments[hospitalId] ?? const [],
              departmentIds: departmentIds[hospitalId] ?? const {},
              doctors: doctorsByHospital[hospitalId] ?? const [],
              isAvailable:
                  (operatingStatus == 'open' || operatingStatus == 'limited') &&
                  erStatus != 'full' &&
                  erStatus != 'temporarily_closed',
              estimatedWaitMinutes: null,
              availableBeds: _intValue(emergency?['available_beds']),
              totalBeds: _intValue(emergency?['maximum_capacity']),
              latitude: latitude,
              longitude: longitude,
```

Fields are read defensively with null-safe conversions. Operating and emergency states are combined into a meaningful isAvailable value, while geographic coordinates and capacity fields are normalized before the UI receives them.

Invalid responses: PostgREST, Edge Function, authentication, repository, and unexpected exceptions are converted into user-safe messages. Raw errors and credentials are not displayed to end users.

WEEK 5 EVIDENCE 4

Live JSON response sample

Figure 4. Live GET response captured from the configured Supabase hospitals endpoint on August 15, 2026.

Live JSON response sample

GET /rest/v1/hospitals - captured August 15, 2026


```
[
  {
    "id": "20000000-0000-4000-8000-000000000007",
    "hospital_name": "Aurora Memorial Hospital",
    "city": "Baler",
    "province": "Aurora",
    "verification_status": "verified",
    "operating_status": "open"
  },
  {
    "id": "20000000-0000-4000-8000-000000000002",
    "hospital_name": "Bataan General Hospital and Medical Center",
    "city": "Balanga City",
    "province": "Bataan",
    "verification_status": "verified",
    "operating_status": "open"
  },
  {
    "id": "20000000-0000-4000-8000-000000000003",
    "hospital_name": "Bulacan Medical Center",
    "city": "Malolos City",
    "province": "Bulacan",
    "verification_status": "verified",
    "operating_status": "open"
  }
]
```

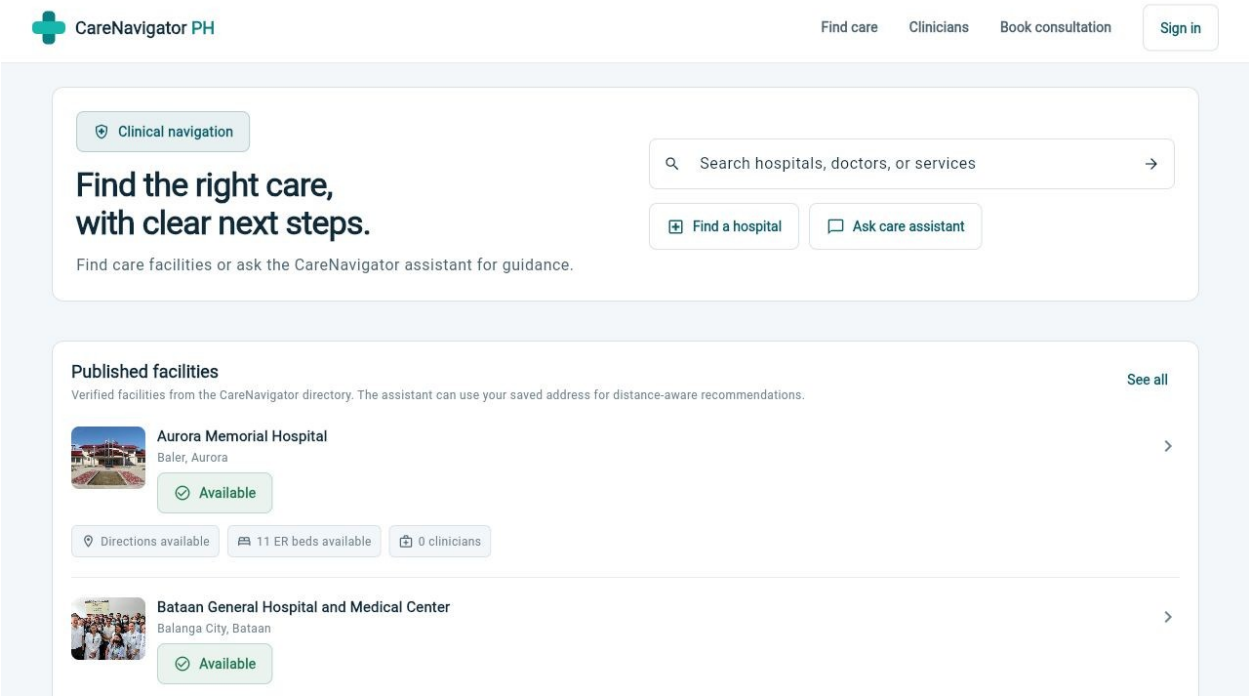

Result: The response returned verified, open facilities. The first two records correspond to the facilities rendered in the application screenshot on the next page.

Security note: the response sample intentionally excludes authentication headers and publishable-key values. Row Level Security remains the authorization boundary for Supabase data access.

WEEK 5 EVIDENCE 5

Retrieved API data displayed in the UI

Figure 5. Running Flutter application displaying hospitals retrieved from the live Supabase API.



Loading and response-state handling

10. Loading: the panel shows “Loading verified facilities” while the repository request is active.
11. Error: a “Hospital directory unavailable” state displays a safe message and Retry button.
12. Empty: “No facilities published” appears when the API returns no matching records.
13. Success: verified facilities are rendered with name, location, operating state, directions, bed availability, and clinician count.

Verification: The live GET response and the rendered UI were captured on August 15, 2026. flutter analyze completed with no issues.

PART II • WEEK 4

Implement CRUD Functionality

Complete Create, Read, Update, and Delete operations using persistent Supabase PostgreSQL storage.

Main data entity: Doctor schedule slots (doctor_schedules). Each record stores the doctor, weekday, start/end time, consultation type, appointment duration, and active state.

CRUD implementation summary

Operation	Repository method	Action	Safeguard
Create	`createScheduleSlot`	INSERT / POST	Validates day, time order, duration, and care mode.
Read	`WorkspaceRepository.load`	SELECT / GET	Filters rows to the authenticated doctor.
Update	`setScheduleActive`	UPDATE / PATCH	Requires a non-empty schedule ID.
Delete	`deleteScheduleSlot`	DELETE	Blocks removal when an appointment depends on the slot.

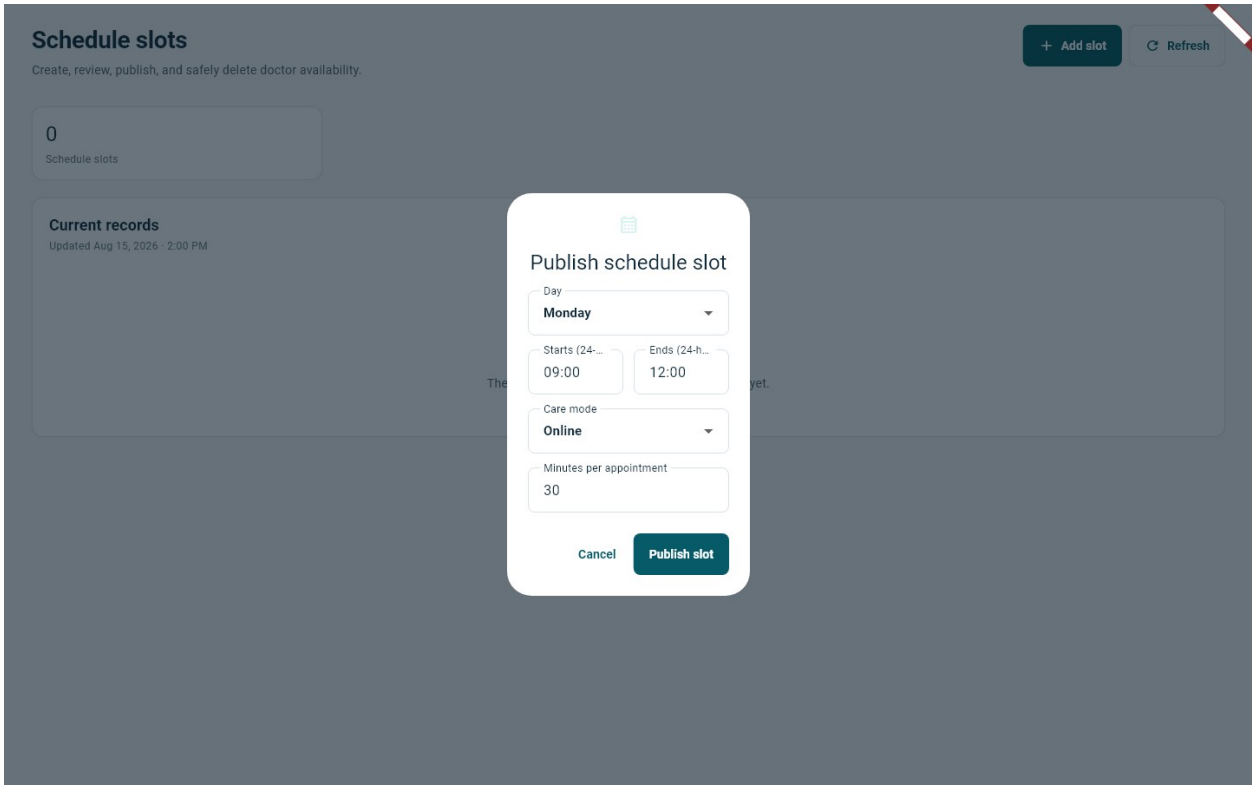
Persistence and user experience

14. Records are persisted in Supabase PostgreSQL and protected by authentication and Row Level Security.
15. The UI provides labeled forms, status chips, publish/unpublish actions, confirmation dialogs, and retry states.
16. Destructive deletion is guarded by a booked-appointment dependency check.

WEEK 4 EVIDENCE 1

Create operation

Figure 6. Add Slot form for creating a doctor schedule record.



The Create form collects a weekday, 24-hour start/end times, care mode, and minutes per appointment. Submitting calls the repository insert and refreshes the persisted schedule list.

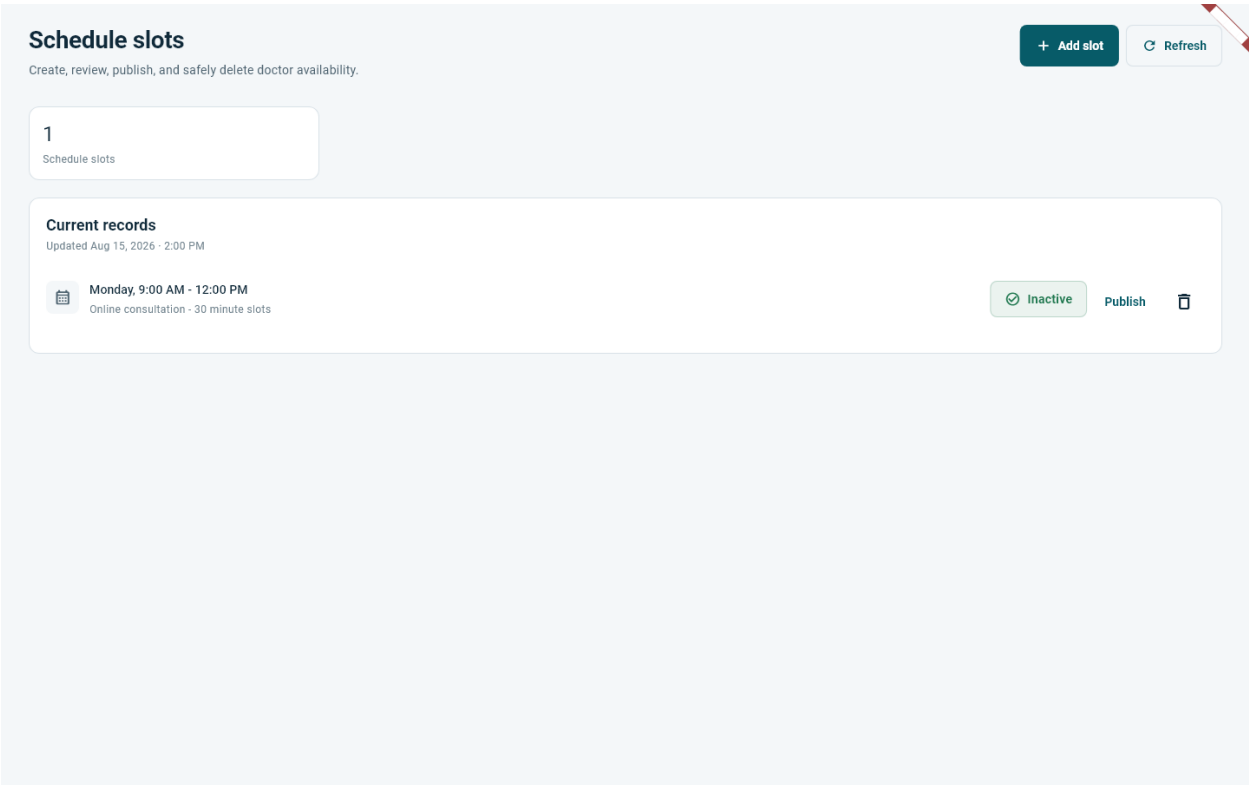
```
CareRepository.createScheduleSlot(...) → doctor_schedules.insert({...})
```

Validation: Day must be 0-6, times must use HH:mm, end must be later than start, duration must be 10-240 minutes, and consultation type is normalized.

WEEK 4 EVIDENCE 2

Read operation

Figure 7. Persisted schedule record displayed in the doctor workspace.



The Read flow selects doctor_schedules through WorkspaceRepository, limits records to the authenticated doctor, orders them, maps JSON rows to WorkspaceItem models, and annotates booking dependencies.

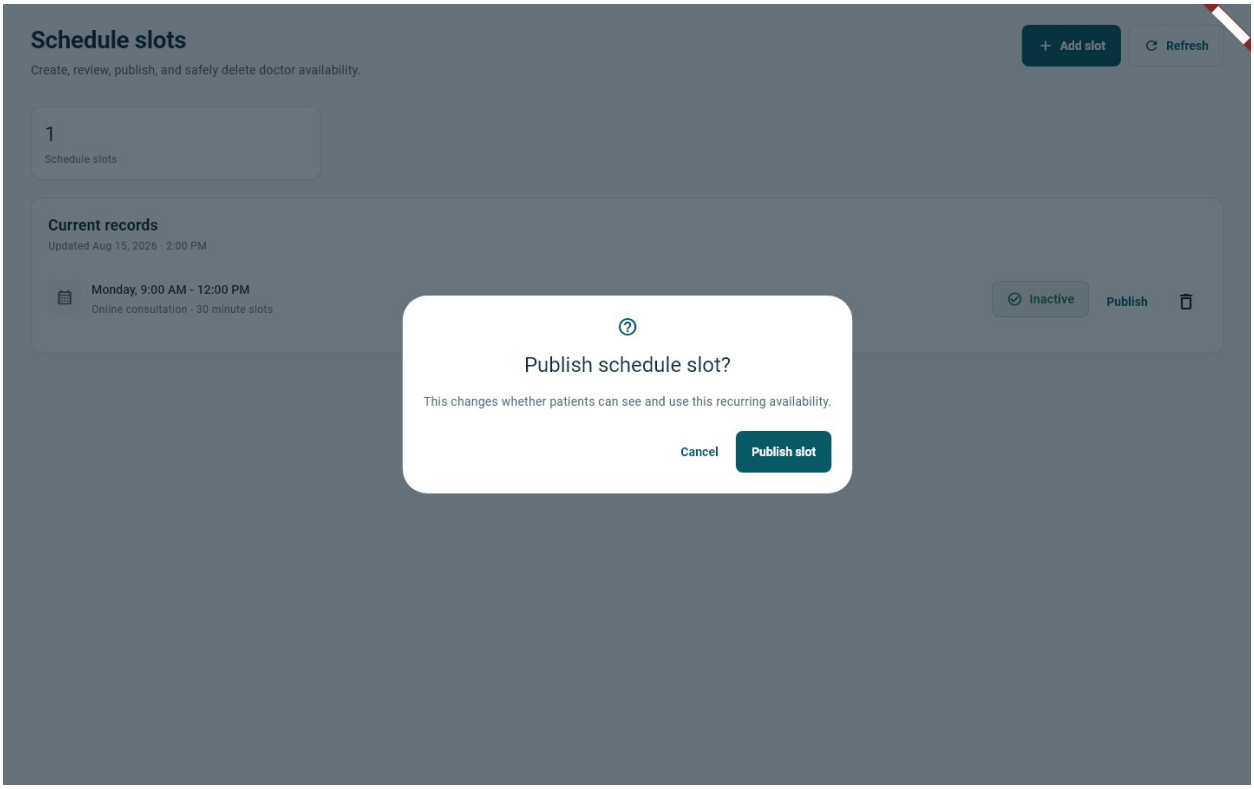
```
WorkspaceRepository.load(...) → from('doctor_schedules').select(...)
```

Meaningful display: The UI shows the weekday, time range, care mode, slot duration, active state, and available row actions instead of raw JSON.

WEEK 4 EVIDENCE 3

Update operation

Figure 8. Confirmation before publishing an inactive schedule slot.



The Update flow toggles is_active for the selected schedule ID. A confirmation dialog explains that publishing changes whether patients can see and use the recurring availability.

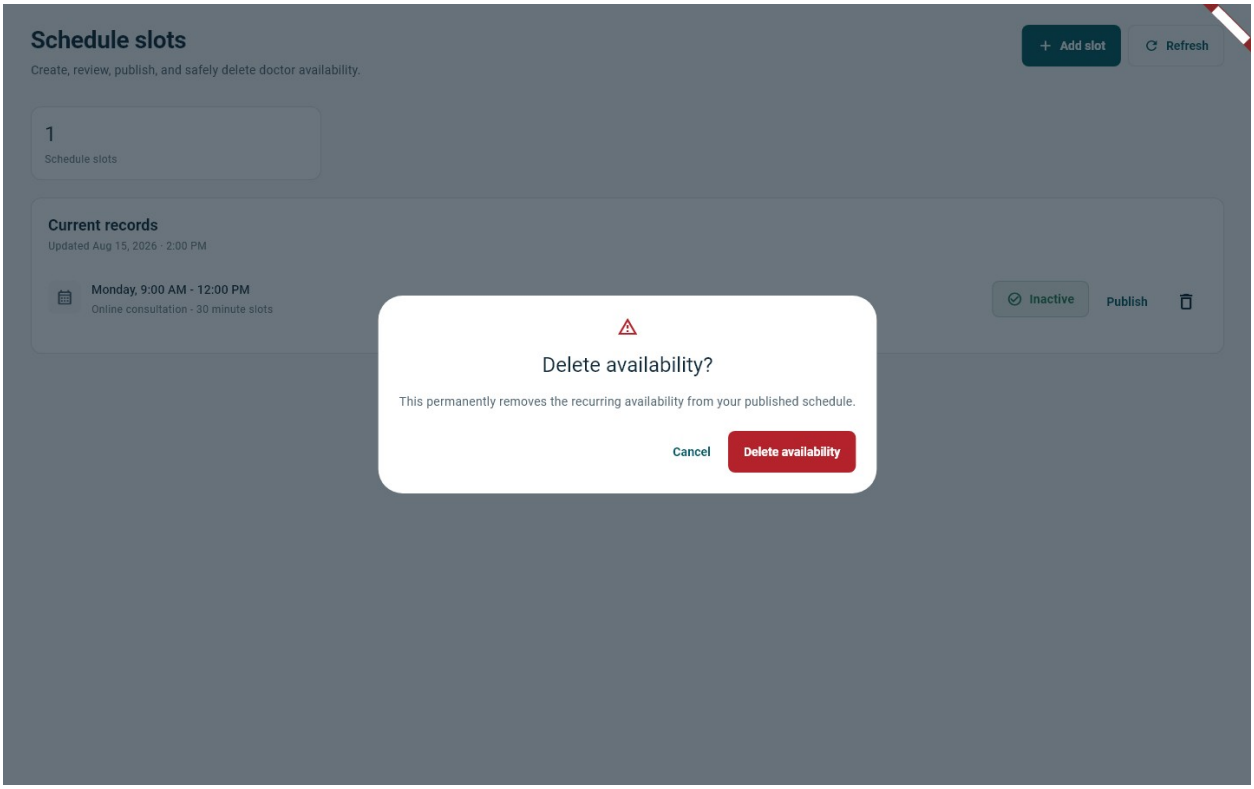
```
CareRepository.setScheduleActive(...) → update({'is_active': active})
```

Safety: The repository rejects an empty schedule ID, updates only the matching row, and requests the updated ID so a missing/invalid record does not silently succeed.

WEEK 4 EVIDENCE 4

Delete operation

Figure 9. Destructive-action confirmation for deleting availability.



The Delete flow first confirms ownership, compares the recurring slot with non-cancelled appointments, and removes the record only when no booking depends on it.

```
CareRepository.deleteScheduleSlot(...) → delete().eq('id', scheduleId)
```

Protected deletion: If a booked consultation matches the slot, deletion is blocked with a clear message. The final confirmation uses explicit destructive wording.

WEEK 4 DOCUMENTATION

Validation, persistence, and verification

How CRUD was implemented

The Flutter UI delegates all database work to typed repository classes. Supabase translates select, insert, update, and delete calls into HTTP requests to PostgreSQL through PostgREST. Riverpod reloads the workspace snapshot after successful mutations, so users immediately see persisted state.

Data validation

17. Client validation prevents malformed day, time, duration, care-mode, and identifier values before a network request.
18. Database constraints and RLS provide server-side persistence and access control.
19. Delete checks protect appointments that depend on a recurring slot.
20. Repository failures are converted into concise messages suitable for the UI.

Verification results

Static analysis: flutter analyze: No issues found (August 15, 2026).

Focused tests: 30 repository-validation, primary-action, row-action, and CRUD-evidence tests passed. The four screenshots in this section were generated by a reproducible Flutter widget test without modifying live medical data.

Submission checklist

21. Updated project source code: present in the CareNavigator PH workspace.
22. API request implementation screenshots: Figures 1 and 2.
23. Retrieved API data screenshot: Figure 5.
24. JSON response sample: Figure 4.
25. Create, Read, Update, and Delete screenshots: Figures 6-9.
26. Brief API and CRUD documentation: included in Parts I and II.

End of submission document.